



Propuesta de Caso de Uso

Sistemas y Servicios en la Nube (2025/26)

Autor: Raúl Jiménez de la Cruz | raul.jimenez8@alu.uclm.es

Fecha: 26 de mayo de 2026

Índice general

1	Introducción	1
2	Diseño de Componentes	2
2.1	CCM Component Design	2
2.2	CCM Architecture Design	2
	Referencias	3
A	Diagrama de Arquitectura	4
A.1	Asignación de cada Componente Lógico al Recurso AWS	5

SECCIÓN 1

Introducción

La siguiente propuesta de caso de uso tiene como principal objetivo envolver un microservicio de Java en una arquitectura AWS profesional, que integre múltiples servicios para dotarlo de diferentes funcionalidades.

Para la ocasión, se utilizará como base una aplicación Java, desarrollada en un marco Spring Boot, que sigue las normas básicas del programa de televisión “Cifras y Letras”, concretamente la sección “La cifra exacta”. El objetivo es, dados seis números del 1 al 9, 25, 50, 75 y 100 y un objetivo a alcanzar –un número entre el 100 y el 999 generalmente–, encontrar la secuencia de operaciones necesarias para alcanzar ese objetivo o, si no es posible, quedarse lo más cerca posible.

El funcionamiento del microservicio se basa en pasar mediante argumentos de maven los números para jugar, y la salida del microservicio es la secuencia de operaciones necesarias para alcanzar el objetivo o, si no es posible, la secuencia más cercana posible. Se proporcionará un ejemplo a continuación:

```
$ mvn spring-boot:run -Dspring-boot.run.arguments="100 75 25 2 3 1
301"
...
Target Number: 301
Numbers to use: 100 75 25 2 3 1
[1] Best solution found with distance 0 and 5 operations:
100 + 75 = 175
25 + 1 = 26
175 - 26 = 149
2 * 149 = 298
3 + 298 = 301
```

En este momento, el microservicio es capaz de encontrar el exacto, en caso de que exista, pero no siempre encuentra la ruta más óptima. Este supuesto será tomado en cuenta para futuras versiones, aunque **no es el objetivo de la presente práctica.**

SECCIÓN 2

Diseño de Componentes

2.1. CCM COMPONENT DESIGN

Componente	Capa	Tipo CCM	Rol funcional	Interfaz expuesta	Dependencias
Web UI	Frontend	Presentation component	Selección de números y cifra objetivo. Visualización de resultados.	HTML/JS estático servido vía HTTP	CDN Delivery, Auth Service, API Endpoint
CDN Delivery	Frontend	Content delivery component	Distribución global del frontend estático con caché.	HTTPS (edge)	Static Storage
Static Storage	Frontend	Storage component	Almacén de los assets estáticos del frontend.	S3 Object API (privada, solo CDN)	—
Auth Service	Security	Identity component	Registro, login y emisión de tokens JWT para usuarios.	OAuth 2.0 / OIDC	—
Firewall	Security	Security component	Filtrado de tráfico malicioso y <i>rate limiting</i> sobre la API.	Inline (transparente al cliente)	API Endpoint
API Endpoint	Gateway	API gateway component	Exposición REST del backend. Valida token JWT (Cognito authorizer).	HTTPS REST	Auth Service, Message Queue
Message Queue	Logic	Messaging component	Buffer asíncrono entre API y lógica de negocio. Reintentos y DLQ.	AMQP-like (SQS API)	Solver Engine
Solver Engine	Logic	Processing component	Núcleo de cálculo. Resuelve la cifra exacta con los dígitos dados.	Invocación por evento (SQS trigger)	Game Repository, Notification Hub
Game Repository	Data	Data store component	Persistencia de partidas y resultados por usuario.	NoSQL API (DynamoDB)	—
Notification Hub	Notify	Messaging component	Enrutamiento de notificaciones hacia uno o varios canales.	Pub/Sub (topic ARN)	Email Dispatcher
Email Dispatcher	Notify	Notification component	Envío de resultados y digest semanal por correo.	SMTP / SES API	—
Scheduler	Ops	Scheduling component	Dispara el digest semanal de resultados de forma periódica.	Cron expression	Notification Hub
Observability	Ops	Monitoring component	Métricas, trazas distribuidas y alertas de toda la pila.	Dashboard / Alerting API	Todos los componentes

2.2. CCM ARCHITECTURE DESIGN

El diagrama de arquitectura completo se muestra en el Apéndice Diagrama de Arquitectura.

Referencias

The LaTeX Project. (2008, mayo). *The latex project public license, version 1.3c*. The LaTeX Project. Descargado de <https://www.latex-project.org/lppl/lppl-1-3c.txt>

ANEXO A:

Diagrama de Arquitectura



Figura A.1: Diagrama de Arquitectura

A.1. ASIGNACIÓN DE CADA COMPONENTE LÓGICO AL RECURSO AWS

Componente	Capa	Servicio AWS	Recurso / configuración clave	Recurso Terraform principal	Notas de despliegue	Prioridad
Web UI	Frontend	S3	Bucket con <i>static website hosting</i> , acceso solo desde OAC	aws_s3_bucket	Build del frontend desplegado con aws_s3_object o CI/CD externo	2
CDN Delivery	Frontend	CloudFront	Distribution con OAC, certificado ACM (<i>us-east-1</i>), HTTPS forzado	aws_cloudfront_distribution	Provider alias <i>us-east-1</i> obligatorio para ACM	2
Static Storage	Frontend	S3	Mismo bucket que Web UI; acceso bloqueado públicamente	aws_s3_bucket_public_access_block	Separar bucket de assets de bucket de logs	2
Auth Service	Security	Cognito	User Pool + App Client + dominio propio	aws_cognito_user_pool	JWT emitido es validado por API Gateway authorizer	3
Firewall	Security	AWS WAF v2	WebACL REGIONAL asociada a API Gateway; regla de <i>rate limiting</i>	aws_wafv2_web_acl	Scope REGIONAL para API GW; CLOUDFRONT requeriría <i>us-east-1</i>	4
API Endpoint	Gateway	API Gateway v2 (HTTP)	Ruta POST /solve, JWT authorizer (Cognito), integración SQS	aws_apigatewayv2_api	Integración directa API GW → SQS sin pasar por Lambda	3
Message Queue	Logic	SQS	Cola estándar + DLQ; visibility timeout alineado con timeout de Lambda	aws_sqs_queue	DLQ con alarma CloudWatch en <code>ApproximateNumberOfMessagesNotVisible</code>	2
Solver Engine	Logic	Lambda	Runtime Java 21, imagen Docker en ECR, <i>event source mapping</i> SQS	aws_lambda_function	SnapStart activado para reducir <i>cold start</i> del JAR Spring Boot	1
Game Repository	Data	DynamoDB	PK: <code>userId</code> , SK: <code>timestamp</code> ; GSI por resultado para ranking	aws_dynamodb_table	TTL opcional para limpiar partidas antiguas automáticamente	1
Notification Hub	Notify	SNS	Topic estándar; suscripción SES como endpoint	aws_sns_topic	Permite añadir canales futuros (SMS, webhook) sin tocar Lambda	2
Email Dispatcher	Notify	SES	Identidad de dominio verificada, DKIM configurado	aws_ses_domain_identity	Verificación DNS es paso manual fuera de Terraform	2
Scheduler	Ops	EventBridge Scheduler	Cron semanal; target: SNS topic del Notification Hub	aws_scheduler_schedule	Rol IAM con permiso <code>sns:Publish</code> sobre el topic ²	4
Observability	Ops	CloudWatch + X-Ray	Log groups con retención, dashboards, alarmas, X-Ray activo en Lambda	aws_cloudwatch_log_group	X-Ray se activa con <code>tracing_config { mode = "Active" }</code> en Lambda	3

¹La columna *prioridad* indica el orden de implementación en AWS, siendo 1 más esencial o prioritario, y 4 como mejoras opcionales que no rompen los artefactos que tienen una mayor prioridad

²Dado el entorno AWS Academy Learner Lab, el rol será `LabRole`